LINUX OPS FIELDBOOK

Production Troubleshooting & Performance Guide

```
$ sudo systemctl status
$ tail -f /var/log/syslog
$ netstat -tulpn
$ iostat -x 1
```

Bo Wang

v1.0 | 2026 Edition

Linux Ops Fieldbook

生产环境排障与性能优化实战手册

RHEL 8/9 | Rocky | Alma | Oracle Linux

Bo Wang (王波)

Oracle 新西兰首席技术支持工程师

v0279158888@gmail.com
github.com/bowang168 | linkedin.com/in/bowang168

v1.0 | 2026 Edition

Contents

1 Linux Ops Fieldbook 8
1.1 生产环境排障与性能优化实战手册 8
1.2 关于作者 8
1.3 目录 8

2 第一章系统启动与恢复 10
2.1 1.1 启动流程概览 10
2.2 1.2 GRUB2 管理 10
2.2.1 查看与修改默认启动项 10
2.2.2 重新生成 GRUB 配置 10
2.2.3 GRUB2 密码保护 11
2.3 1.3 启动故障恢复速查 11
2.4 1.4 initramfs 管理 11
2.5 1.5 systemd 启动目标 12

3 第二章内核管理与调优 13
3.1 2.1 内核版本管理 13
3.2 2.2 内核模块管理 13
3.3 2.3 sysctl 内核参数调优 14
3.3.1 常用调优参数速查 14
3.4 2.4 Huge Pages 与 THP 14
3.5 2.5 UDEV 规则 15
3.6 2.6 Kdump 崩溃转储 15
3.7 2.7 Magic SysRq 紧急操作 16

4 第三章存储与文件系统 17
4.1 3.1 块设备与分区 17
4.1.1 分区工具对比 17
4.2 3.2 LVM 逻辑卷管理 17
4.2.1 概念层次 17
4.2.2 核心操作 17
4.2.3 LVM 灾难恢复 18
4.3 3.3 文件系统 18
4.3.1 对比 18
4.3.2 常用操作 19
4.3.3 /etc/fstab 最佳实践 19
4.4 3.4 Multipath 多路径 19
4.4.1 配置 `/etc/multipath.conf` 20
4.4.2 常用命令 20
4.4.3 排错 20
4.5 3.5 NVMe 20
4.6 3.6 Software RAID (mdadm) 21
4.7 3.7 iSCSI 21
4.8 3.8 LUKS 磁盘加密 21
4.9 3.9 磁盘配额 22

5 第四章网络配置与诊断 23
5.1 4.1 NetworkManager (nmcli) . . . 23
5.2 4.2 Network Bonding . . . 23
5.3 4.3 VLAN . . . 24
5.4 4.4 防火墙 (firewalld) . . . 24
5.5 4.5 DNS . . . 24
5.5.1 客户端诊断 . . . 24
5.5.2 BIND 服务端 . . . 25
5.6 4.6 tcpdump 抓包分析 . . . 25
5.6.1 TCP Flags 参考 . . . 25
5.7 4.7 网络性能调优 . . . 25
5.7.1 TCP/IP 内核参数 . . . 25
5.7.2 Jumbo Frames . . . 26
5.7.3 网络诊断工具 . . . 26

6 第五章安全加固与合规 27
6.1 5.1 SSH 安全加固 . . . 27
6.1.1 推荐配置 (`/etc/ssh/sshd_config`) . . . 27
6.1.2 密钥管理 . . . 27
6.2 5.2 SELinux . . . 28
6.3 5.3 文件权限 . . . 28
6.3.1 基础权限 . . . 28
6.3.2 ACL (访问控制列表) . . . 29
6.3.3 chattr 扩展属性 . . . 29
6.4 5.4 系统审计 (auditd) . . . 29
6.5 5.5 用户与特权管理 . . . 30
6.5.1 Sudo 最佳实践 . . . 30
6.6 5.6 安全加固清单 . . . 30

7 第六章核心服务管理 32
7.1 6.1 systemd 服务管理 . . . 32
7.1.1 自定义 systemd 服务 . . . 32
7.2 6.2 Chrony 时间同步 . . . 33
7.3 6.3 Rsyslog & journald 日志 . . . 33
7.3.1 Rsyslog . . . 33
7.3.2 journald . . . 33
7.3.3 logrotate . . . 33
7.4 6.4 Web 服务器 . . . 34
7.4.1 Apache . . . 34
7.4.2 Nginx (替代方案) . . . 34
7.5 6.5 NFS . . . 35
7.6 6.6 Samba (CIFS) . . . 35
7.7 6.7 LDAP / AD 集成 . . . 36
7.8 6.8 KVM 虚拟化 . . . 36
7.9 6.9 Postfix 邮件 . . . 36

8 第七章容器与云原生 38
8.1 7.1 为什么是 Podman? 38
8.2 7.2 容器基本操作 38
8.3 7.3 容器镜像构建 39
8.3.1 Containerfile (等同于 Dockerfile) 39
8.3.2 镜像最佳实践 39
8.4 7.4 容器网络 40
8.5 7.5 容器存储 40
8.6 7.6 Rootless 容器 41
8.7 7.7 systemd 集成 41
8.8 7.8 Podman Compose 41
8.9 7.9 容器排障 42
8.10 7.10 Kubernetes 基础排障 42

9 第八章 Shell 与自动化 44
9.1 8.1 Bash 核心速查 44
9.1.1 变量与参数 44
9.1.2 特殊变量 44
9.1.3 条件测试 44
9.1.4 I/O 重定向 45
9.2 8.2 循环与函数 45
9.3 8.3 生产脚本模板 45
9.4 8.4 grep 46
9.5 8.5 find 46
9.6 8.6 awk 46
9.7 8.7 sed 47
9.8 8.8 实用管道组合 47
9.9 8.9 Ansible 快速入门 47
9.9.1 Playbook 示例 48

10 第九章性能分析与调优 49
10.1 9.1 性能分析方法论 49
10.1.1 USE 方法 (Brendan Gregg) 49
10.1.2 快速排障流程 49
10.2 9.2 CPU 分析 49
10.3 9.3 内存分析 50
10.4 9.4 磁盘 IO 分析 50
10.4.1 iostat 关键指标 50
10.4.2 IO 调度器 51
10.5 9.5 TuneD 自动调优 51
10.6 9.6 资源限制 52
10.6.1 ulimits 52
10.6.2 cgroups v2 52

11 第十章诊断工具与排障方法论 53
11.1 10.1 Sosreport 53
11.2 10.2 PCP (Performance Co-Pilot) 53
11.3 10.3 eBPF 工具 (bcc / bpftrace) 53
11.4 10.4 strace 系统调用跟踪 54
11.5 10.5 排障方法论 54

11.5.1 五步排障法 54
11.5.2 常见故障排查路径 54
11.6 10.6 日志分析速查 55

12 附录 A 速查表 56
12.1 A.1 systemctl 速查 56
12.2 A.2 文件系统操作速查 56
12.3 A.3 网络诊断速查 56
12.4 A.4 性能工具速查 57
12.5 A.5 容器速查 57
12.6 版权声明 57

Chapter 1

Linux Ops Fieldbook

1.1 生产环境排障与性能优化实战手册

作者：Bo Wang (王波) 版本：v1.0 | 2026 Edition

“在生产环境中，你不需要知道所有事情，但你需要在 5 分钟内知道去哪里查。”

本书面向有一定 Linux 基础的运维工程师、SRE、DevOps 工程师和技术支持人员。内容覆盖从系统启动到容器编排、从磁盘排障到性能调优的全链路知识体系。所有命令和方案均基于 **RHEL 8/9 及兼容发行版 (Rocky, Alma, Oracle Linux)**，适用于 2024—2030 年主流生产环境。

1.2 关于作者

Bo Wang (王波) —Oracle 新西兰首席技术支持工程师，拥有 17 年企业级 IT 经验，先后任职于 IBM 和 Oracle。持有 RHCE、CCNP、PMP、OCI 等多项专业认证。在 Linux 内核调优、云基础设施 (OCI, AWS)、存储架构、网络性能优化和大规模故障排查（1,200+ 生产环境）方面有深厚的实战积累。本书源自作者多年一线支持工作中总结的排障笔记和最佳实践，经过系统化整理后面向社区分享。

- 邮箱: v0279158888@gmail.com
- GitHub: github.com/bowang168
- LinkedIn: linkedin.com/in/bowang168
- 博客: bowang168.github.io

1.3 目录

- 第一章系统启动与恢复
- 第二章内核管理与调优
- 第三章存储与文件系统
- 第四章网络配置与诊断
- 第五章安全加固与合规
- 第六章核心服务管理

- 第七章容器与云原生
- 第八章 Shell 与自动化
- 第九章性能分析与调优
- 第十章诊断工具与排障方法论
- 附录 A 速查表

Chapter 2

第一章系统启动与恢复

本章目标：掌握 Linux 启动全流程，能在 GRUB 损坏、内核 panic、忘记 root 密码等场景下快速恢复系统。

2.1 1.1 启动流程概览

现代 Linux 系统的启动顺序如下：

固件 (UEFI/BIOS) → GRUB2 → Kernel → initramfs → systemd → 多用户目标

阶段	组件	关键文件/位置
固件初始化	UEFI / Legacy BIOS	NVRAM / MBR
引导加载	GRUB2	`/boot/grub2/grub.cfg`
内核加载	vmlinuz	`/boot/vmlinuz-*`
初始文件系统	initramfs	`/boot/initramfs-*.img`
系统初始化	systemd (PID 1)	`/etc/systemd/system/`

2.2 1.2 GRUB2 管理

2.2.1 查看与修改默认启动项

```
# 查看当前默认
grub2-editenv list

# 列出可用内核
grubby --info=ALL | grep ^kernel

# 设置默认内核
grub2-set-default 0
# 或指定内核版本
grubby --set-default /boot/vmlinuz-5.14.0-427.el9.x86_64
```

2.2.2 重新生成 GRUB 配置

```
# BIOS 系统
grub2-mkconfig -o /boot/grub2/grub.cfg

# UEFI 系统
grub2-mkconfig -o /boot/efi/EFI/redhat/grub.cfg
```

2.2.3 GRUB2 密码保护

防止未授权用户通过 GRUB 修改启动参数获取 root 权限：

```
grub2-setpassword
# 密码哈希存储在 /boot/grub2/user.cfg
```

2.3 1.3 启动故障恢复速查

故障场景	恢复步骤
忘记 root 密码	GRUB 编辑启动行 → 添加 `rd.break` → `chroot /sysroot` → `passwd root` → `touch /.autorelabel`
内核 panic	GRUB 选择上一个可用内核启动，修复后 `grubby --set-default`
initramfs 损坏	Rescue 模式 → `dracut -f /boot/initramfs-$(uname -r).img $(uname -r)`
GRUB 被覆盖	Rescue 介质 → `chroot /mnt/sysimage` → `grub2-install /dev/sda`
UEFI 启动项丢失	`efibootmgr -c -d /dev/sda -p 1 -L "Linux" -l "\EFI\redhat\shimx64.efi"`
进入 Emergency 模式	检查 `/etc/fstab` 中的挂载项是否正确，`systemctl default` 恢复

2.4 1.4 initramfs 管理

```
# 重建当前内核的 initramfs
dracut -f

# 重建所有内核
dracut -f --regenerate-all

# 仅包含当前主机需要的驱动（精简版）
dracut --hostonly -f

# 查看 initramfs 内容
lsinitrd /boot/initramfs-$(uname -r).img | head -50

# 添加额外模块到 initramfs
dracut --add-drivers "mpt3sas megaraid_sas" -f
```

2.5 1.5 systemd 启动目标

```
# 查看当前目标
systemctl get-default

# 切换目标
systemctl set-default multi-user.target      # 命令行
systemctl set-default graphical.target       # 图形界面

# 紧急模式
systemctl isolate rescue.target
systemctl isolate emergency.target

# 查看启动耗时
systemd-analyze
systemd-analyze blame | head -20
systemd-analyze critical-chain
```

Chapter 3

第二章内核管理与调优

本章目标：掌握内核版本管理、模块操作、sysctl 参数调优和 Kdump 配置。

3.1 2.1 内核版本管理

```
# 查看当前内核
uname -r

# 列出已安装内核
rpm -qa kernel-core | sort

# 安装新内核
dnf install kernel

# 设置默认内核
grubby --set-default /boot/vmlinuz-<version>

# 删除旧内核（保留最近 2 个）
dnf remove --oldinstallonly --setopt installonly_limit=2 kernel
```

3.2 2.2 内核模块管理

```
# 查看已加载模块
lsmod

# 查看模块详情
modinfo e1000e

# 加载 / 卸载模块
modprobe bonding
modprobe -r bonding

# 持久化加载
echo "bonding" > /etc/modules-load.d/bonding.conf

# 黑名单（禁用模块）
cat > /etc/modprobe.d/blacklist-nouveau.conf << EOF
blacklist nouveau
options nouveau modeset=0
```

```
EOF
dracut -f   # 重建 initramfs 使黑名单生效
```

3.3 2.3 sysctl 内核参数调优

```
# 查看所有参数
sysctl -a | wc -l    # 通常 1000+ 个参数

# 临时修改
sysctl -w vm.swappiness=10

# 持久化
cat > /etc/sysctl.d/99-tuning.conf << EOF
# 内存管理
vm.swappiness = 10
vm.dirty_ratio = 15
vm.dirty_background_ratio = 5

# 网络优化
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 65535
net.ipv4.ip_local_port_range = 1024 65535

# 文件句柄
fs.file-max = 2097152
fs.inotify.max_user_watches = 524288
EOF

sysctl -p /etc/sysctl.d/99-tuning.conf
```

3.3.1 常用调优参数速查

参数	默认值	推荐值	说明
vm.swappiness	60	10	降低 swap 使用倾向 (数据库服务器建议 1)
vm.dirty_ratio	20	15	脏页占内存比例上限
vm.overcommit_memory	0	0	0= 启发式, 1= 总是允许, 2= 严格限制
net.core.somaxconn	4096	65535	TCP 监听队列最大长度
net.ipv4.tcp_tw_reuse	2	1	允许 TIME_WAIT 重用
fs.file-max	系统依赖	2097152	系统级最大文件描述符

3.4 2.4 Huge Pages 与 THP

```
# 查看 THP (Transparent Huge Pages) 状态
cat /sys/kernel/mm/transparent_hugepage/enabled
```

```
# 关闭 THP（数据库场景推荐）
echo never > /sys/kernel/mm/transparent_hugepage/enabled
echo never > /sys/kernel/mm/transparent_hugepage/defrag

# 持久化：在 GRUB 内核参数中添加
# transparent_hugepage=never

# 静态 Huge Pages（Oracle DB / SAP HANA 场景）
sysctl -w vm.nr_hugepages=1024
echo "vm.nr_hugepages = 1024" >> /etc/sysctl.d/99-hugepages.conf
grep Huge /proc/meminfo
```

3.5 2.5 UDEV 规则

```
# 查看设备属性（用于编写规则）
udevadm info -a -n /dev/sdb

# 自定义规则示例：磁盘持久命名
cat > /etc/udev/rules.d/99-oracle-disks.rules << EOF
ACTION=="add|change", KERNEL=="sd*", ENV{ID_SERIAL}=="36001405...", SYMLINK+="oracleasm/disk1"
EOF

# 重载规则
udevadm control --reload-rules
udevadm trigger
```

3.6 2.6 Kdump 崩溃转储

```
# 安装和配置
dnf install kexec-tools
systemctl enable --now kdump

# 配置 /etc/kdump.conf
path /var/crash
core_collector makedumpfile -l --message-level 1 -d 31

# 验证 Kdump 是否活跃
kdumpctl status

# □  测试（会立即触发系统崩溃重启）
echo 1 > /proc/sys/kernel/sysrq
echo c > /proc/sysrq-trigger

# 分析 vmcore
crash /usr/lib/debug/lib/modules/$(uname -r)/vmlinux \
      /var/crash/<timestamp>/vmcore
```

3.7 2.7 Magic SysRq 紧急操作

当系统完全卡死时的最后手段：

```
# 启用 SysRq
echo 1 > /proc/sys/kernel/sysrq
```

安全重启序列 **R–E–I–S–U–B**（“Reboot Even If System Utterly Broken”）：

键	功能
R	从 X 收回键盘控制权
E	向所有进程发送 SIGTERM
I	向所有进程发送 SIGKILL
S	同步所有文件系统
U	以只读模式重新挂载所有文件系统
B	立即重启

```
# 远程触发（每步间隔几秒）
echo s > /proc/sysrq-trigger
echo u > /proc/sysrq-trigger
echo b > /proc/sysrq-trigger
```

Chapter 4

第三章存储与文件系统

本章目标：掌握分区、LVM、多路径、NVMe、RAID、iSCSI、加密等存储技术的配置与排障。

4.1 3.1 块设备与分区

4.1.1 分区工具对比

工具	分区表类型	最大支持	适用场景
`fdisk`	MBR	2 TiB	传统 BIOS 系统
`gdisk`	GPT	8 ZiB	UEFI 系统、大容量磁盘
`parted`	MBR + GPT	无限制	脚本化分区操作

```
# 查看块设备
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL

# GPT 分区（推荐）
parted -s -a optimal /dev/sdb mklabel gpt \
  mkpart primary 0% 100%

# 通知内核分区表更新
partprobe /dev/sdb

# 获取磁盘 UUID 和 WWID
blkid /dev/sdb1
/lib/udev/scsi_id --whitelisted --device=/dev/sdb
```

4.2 3.2 LVM 逻辑卷管理

4.2.1 概念层次

物理磁盘 → PV（物理卷）→ VG（卷组）→ LV（逻辑卷）→ 文件系统

4.2.2 核心操作

```
# === 创建 ===
pvcreate /dev/sdb1 /dev/sdc1
vgcreate vg_data /dev/sdb1 /dev/sdc1
```

```
lvcreate -L 200G -n lv_app vg_data
mkfs.xfs /dev/vg_data/lv_app
mount /dev/vg_data/lv_app /app

# === 扩展（在线）===
# 扩展 VG（添加新磁盘）
pvcreate /dev/sdd1
vgextend vg_data /dev/sdd1

# 扩展 LV + 文件系统
lvextend -L +100G /dev/vg_data/lv_app
xfs_growfs /app          # XFS（只能在线扩展）
resize2fs /dev/vg_data/lv_app   # Ext4

# === 快照 ===
lvcreate -L 1G -s -n lv_app_snap /dev/vg_data/lv_app
mount -o ro /dev/vg_data/lv_app_snap /mnt/snap

# === 查看 ===
pvs && vgs && lvs
lvs -o +devices          # 查看 LV 所在物理设备
```

4.2.3 LVM 灾难恢复

```
# 备份元数据（自动保存在 /etc/lvm/backup/）
vgcfgbackup vg_data

# 恢复元数据
vgcfgrestore -f /etc/lvm/backup/vg_data vg_data
vgchange -ay vg_data

# 修复 "unknown device" PV
pvs -o pv_name,vg_name,pv_uuid    # 获取 UUID
pvcreate --restorefile /etc/lvm/archive/<vg_backup> \
  --uuid <original_uuid> /dev/new_disk
vgcfgrestore -f /etc/lvm/archive/<vg_backup> vg_data
vgchange -ay vg_data
```

4.3 3.3 文件系统

4.3.1 对比

特性	XFS	Ext4	Btrfs
RHEL/OL 默认	8/9	6/7	仅实验性
最大卷大小	8 EiB	1 EiB	16 EiB
在线扩展			

特性	XFS	Ext4	Btrfs
在线缩小			
快照			原生
适用场景	大文件/高吞吐	通用	开发/实验

4.3.2 常用操作

```
# === XFS ===
mkfs.xfs /dev/sdb1
xfs_growfs /mountpoint        # 扩展（在线）
xfs_repair /dev/sdb1          # 修复（需卸载）
xfs_info /mountpoint

# === Ext4 ===
mkfs.ext4 /dev/sdb1
resize2fs /dev/sdb1           # 扩展
e2fsck -f /dev/sdb1           # 修复（需卸载）
tune2fs -l /dev/sdb1          # 查看超级块

# === inode 耗尽排查 ===
df -i                         # 查看 inode 使用率
find / -xdev -printf '%h\n' | sort | uniq -c | sort -rn | head
```

4.3.3 /etc/fstab 最佳实践

```
# 使用 UUID（避免设备名变化导致挂载失败）
UUID=xxxxxxxx-xxxx  /app  xfs  defaults,noatime  0  0

# 网络存储添加 _netdev
UUID=xxxxxxxx-xxxx  /data  ext4  defaults,_netdev  0  0

# 获取 UUID
blkid /dev/sdb1
```

`/etc/fstab` 配置错误可能导致系统无法启动。修改后建议用 `mount -a` 验证。

4.4 3.4 Multipath 多路径

SAN 存储环境下，同一个 LUN 通过多条物理路径可见，需要多路径软件将其合并为一个逻辑设备。

```
# 安装
dnf install device-mapper-multipath
mpathconf --enable --with_multipathd y
systemctl enable --now multipathd
```

4.4.1 配置 /etc/multipath.conf

```
defaults {
    user_friendly_names yes
    find_multipaths     yes
    failback            immediate
    fast_io_fail_tmo    5
    dev_loss_tmo        30
}

blacklist {
    devnode "^(ram|raw|loop|fd|md|sr|scd|st)[0-9]*"
    devnode "^sd[ab]$"   # 排除本地磁盘
}
```

4.4.2 常用命令

```
multipath -ll                    # 查看拓扑
multipath -f /dev/mapper/mpathX  # 删除 Ghost 设备
multipathd show paths format "%w %d %t %i %o %T %z"
systemctl reload multipathd
```

4.4.3 排错

问题	排查步骤
路径 faulty	`multipath -ll` 查看路径状态，检查 FC/iSCSI 连通性
Ghost 设备	`multipath -f <device>`，重启 multipathd
设备不识别	检查 blacklist 配置，`multipath -v3` 查看详细日志

4.5 3.5 NVMe

```
# 查看 NVMe 设备
nvme list
nvme list-subsys         # 查看多路径拓扑

# 参数调优
grep "" /sys/module/nvme_core/parameters/*
echo 120 > /sys/module/nvme_core/parameters/io_timeout   # 默认 30s

# NVMe over Fabrics (NVMeoF)
nvme discover -t tcp -a 192.168.1.100 -s 4420
nvme connect -t tcp -a 192.168.1.100 -s 4420 -n <subsystem_nqn>
```

4.6 3.6 Software RAID (mdadm)

```
# 创建 RAID 1 (镜像)
mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc

# 监控状态
cat /proc/mdstat
mdadm --detail /dev/md0

# 磁盘故障替换
mdadm --manage /dev/md0 --fail /dev/sdb
mdadm --manage /dev/md0 --remove /dev/sdb
mdadm --manage /dev/md0 --add /dev/sdd

# 保存配置
mdadm --detail --scan >> /etc/mdadm.conf
```

4.7 3.7 iSCSI

```
# Initiator 端配置
dnf install iscsi-initiator-utils
systemctl enable --now iscsid

# 发现目标
iscsiadm -m discovery -t st -p 192.168.1.100

# 登录
iscsiadm -m node -T <target_iqn> -p 192.168.1.100:3260 -l

# 设置自动登录
iscsiadm -m node -T <target_iqn> -o update \
  -n node.startup -v automatic
```

4.8 3.8 LUKS 磁盘加密

```
# 创建加密卷
cryptsetup luksFormat /dev/sdd
cryptsetup luksOpen /dev/sdd secure_data
mkfs.xfs /dev/mapper/secure_data
mount /dev/mapper/secure_data /secure

# 自动挂载
echo "secure_data /dev/sdd none" >> /etc/crypttab
echo "/dev/mapper/secure_data /secure xfs _netdev 0 0" >> /etc/fstab

# 管理密钥槽
cryptsetup luksDump /dev/sdd              # 查看密钥槽
```

```
cryptsetup luksAddKey /dev/sdd            # 添加备用密钥
cryptsetup luksRemoveKey /dev/sdd         # 删除密钥
```

4.9 3.9 磁盘配额

```
# 启用配额 (Ext4)
mount -o remount,usrquota,grpquota /data
quotacheck -cugm /data
quotaon /data
edquota -u <user>                # 编辑软/硬限额
repquota -a                      # 查看使用报告

# XFS 配额
mount -o uquota,gquota /dev/sdb1 /data
xfs_quota -x -c 'limit bsoft=5g bhard=6g user1' /data
xfs_quota -x -c 'report -h' /data
```

Chapter 5

第四章网络配置与诊断

本章目标：掌握网络接口配置、防火墙管理、DNS、抓包分析和网络性能调优。

5.1 4.1 NetworkManager (nmcli)

RHEL 8+ 推荐使用 NetworkManager。RHEL 9 已弃用传统 `ifcfg` 脚本。

```
# 查看状态
nmcli device status
nmcli connection show

# 配置静态 IP
nmcli con mod eth0 ipv4.addresses 192.168.1.10/24
nmcli con mod eth0 ipv4.gateway 192.168.1.1
nmcli con mod eth0 ipv4.dns "8.8.8.8 1.1.1.1"
nmcli con mod eth0 ipv4.method manual
nmcli con up eth0

# DHCP
nmcli con mod eth0 ipv4.method auto
nmcli con up eth0
```

5.2 4.2 Network Bonding

```
# 创建 Bond
nmcli con add type bond ifname bond0 mode active-backup \
  bond.options "miimon=100"
nmcli con add type ethernet ifname eth0 master bond0
nmcli con add type ethernet ifname eth1 master bond0
nmcli con up bond-bond0
```

模式	名称	是否需要交换机配置	场景
0	balance-rr	是 (EtherChannel)	负载均衡
1	**active-backup**	**否**	**高可用 (最常用)**
4	802.3ad (LACP)	是 (LACP)	带宽聚合
6	balance-alb	否	自适应负载

5.3 4.3 VLAN

```
nmcli con add type vlan ifname vlan100 dev eth0 id 100
nmcli con mod vlan100 ipv4.addresses 10.0.100.10/24 \
  ipv4.method manual
nmcli con up vlan100
```

5.4 4.4 防火墙 (firewalld)

```
# 基本操作
firewall-cmd --state
firewall-cmd --list-all
firewall-cmd --get-active-zones

# 开放服务/端口
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --permanent --add-port=30000-31000/tcp

# Zone 管理
firewall-cmd --permanent --zone=trusted --add-source=10.0.0.0/8

# 端口转发
firewall-cmd --permanent --add-forward-port=\
  port=80:proto=tcp:toport=8080

# 富规则 (Rich Rule)
firewall-cmd --permanent --add-rich-rule=\
  'rule family="ipv4" source address="10.0.0.0/8" port port="22" protocol="tcp" accept'

# 生效
firewall-cmd --reload
```

> RHEL 8+ 底层已使用 **nftables**，firewalld 作为前端管理工具。直接编写 iptables 规则虽然仍可用，但新系统推荐使用 firewalld。

5.5 4.5 DNS

5.5.1 客户端诊断

```
# dig (推荐)
dig example.com A +short
dig @8.8.8.8 example.com MX
dig -x 1.2.3.4                # 反向解析

# 解析顺序
```

```
cat /etc/nsswitch.conf | grep hosts
# hosts: files dns myhostname
```

5.5.2 BIND 服务端

```
dnf install bind bind-utils
systemctl enable --now named

# 检查配置
named-checkconf
named-checkzone example.com /var/named/example.com.zone
```

5.6 4.6 tcpdump 抓包分析

```
# 基本抓包
tcpdump -i eth0 -nn port 22

# 保存 pcap 文件 (Wireshark 分析)
tcpdump -i eth0 -w /tmp/capture.pcap -c 5000

# 常用过滤器
tcpdump -i eth0 host 10.0.0.1 and port 443
tcpdump -i eth0 'tcp[tcpflags] & tcp-syn != 0'    # 仅 SYN 包
tcpdump -i eth0 icmp                               # 仅 ICMP
tcpdump -i any -s 0 -A port 80                     # 显示 HTTP 明文
```

5.6.1 TCP Flags 参考

Flag	含义	场景
SYN	发起连接	三次握手第一步
SYN-ACK	确认并同步	三次握手第二步
ACK	确认	数据传输确认
FIN	正常关闭	四次挥手
RST	强制重置	连接异常 / 端口未监听
PSH	立即推送	不在缓冲区等待

5.7 4.7 网络性能调优

5.7.1 TCP/IP 内核参数

```
cat > /etc/sysctl.d/99-network.conf << EOF
# 缓冲区
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
```

```
net.ipv4.tcp_rmem = 4096 87380 16777216
net.ipv4.tcp_wmem = 4096 65536 16777216

# 连接管理
net.core.somaxconn = 65535
net.core.netdev_max_backlog = 65535
net.ipv4.tcp_max_syn_backlog = 65535
net.ipv4.ip_local_port_range = 1024 65535

# TIME_WAIT 优化
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fin_timeout = 15

# 启用 BBR 拥塞控制（推荐）
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
EOF

sysctl -p /etc/sysctl.d/99-network.conf
```

5.7.2 Jumbo Frames

```
# 设置 MTU 9000
nmcli con mod eth0 802-3-ethernet.mtu 9000
nmcli con up eth0

# 端到端验证
ping -M do -s 8972 <target_ip>   # 8972 + 28 header = 9000
```

5.7.3 网络诊断工具

```
ss -tuln                          # 查看监听端口（替代 netstat）
ss -s                             # 连接统计
mtr <target>                      # 持续 traceroute
ip -s link show eth0              # 接口统计（含错误计数）
ethtool -S eth0                   # 网卡详细统计
ethtool -k eth0                   # 查看 offload 特性
```

Chapter 6

第五章安全加固与合规

本章目标：掌握 SSH 加固、SELinux、审计、权限管理和安全加固清单。

6.1 5.1 SSH 安全加固

6.1.1 推荐配置（/etc/ssh/sshd_config）

```
# 认证
PermitRootLogin no
PasswordAuthentication no          # 仅密钥认证
PubkeyAuthentication yes
MaxAuthTries 3
AuthenticationMethods publickey

# 加密算法（移除弱算法）
KexAlgorithms curve25519-sha256,diffie-hellman-group16-sha512
Ciphers aes256-gcm@openssh.com,chacha20-poly1305@openssh.com
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com

# 连接管理
ClientAliveInterval 300
ClientAliveCountMax 2
LoginGraceTime 30
MaxSessions 5

# 访问控制
AllowUsers admin deploy
# 或 AllowGroups sshusers
```

6.1.2 密钥管理

```
# 生成 Ed25519 密钥（推荐）
ssh-keygen -t ed25519 -C "admin@server"

# 分发公钥
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@host

# SSH 跳板机
ssh -J jump-host target-host

# ~/.ssh/config
```

```
Host prod-*
    User admin
    ProxyJump jump-host
    IdentityFile ~/.ssh/id_ed25519
    StrictHostKeyChecking yes
```

6.2 5.2 SELinux

```
# 查看状态
getenforce
sestatus

# 模式切换
setenforce 0   # Permissive (仅记录)
setenforce 1   # Enforcing (强制)
# 永久: /etc/selinux/config → SELINUX=enforcing

# === Context 管理 ===
# 查看
ls -Z /var/www/html

# 修改文件上下文
semanage fcontext -a -t httpd_sys_content_t "/data/www(/.*)?"
restorecon -Rv /data/www

# 端口上下文
semanage port -a -t http_port_t -p tcp 8443
semanage port -l | grep http

# === Boolean ===
getsebool -a | grep httpd
setsebool -P httpd_can_network_connect on

# === 排错 ===
ausearch -m AVC -ts recent
sealert -a /var/log/audit/audit.log
# 自动生成策略
audit2allow -a -M mypolicy
semodule -i mypolicy.pp
```

6.3 5.3 文件权限

6.3.1 基础权限

```
# 数字模式: r=4, w=2, x=1
chmod 755 /opt/script.sh    # rwxr-xr-x
chmod 640 /etc/app.conf     # rw-r-----
```

```
# 特殊权限
chmod u+s /usr/bin/prog      # SUID: 以文件所有者身份执行
chmod g+s /shared            # SGID: 新文件继承目录组
chmod +t /tmp                # Sticky: 仅所有者可删除
```

6.3.2 ACL (访问控制列表)

```
# 设置 ACL
setfacl -m u:deploy:rwx /data/releases
setfacl -m g:devteam:rx /data/releases

# 默认 ACL (新文件自动继承)
setfacl -d -m u:deploy:rwx /data/releases

# 查看 / 删除
getfacl /data/releases
setfacl -b /data/releases       # 删除所有 ACL

# 备份与恢复
getfacl -R /data > /backup/acl.txt
setfacl --restore=/backup/acl.txt
```

6.3.3 chattr 扩展属性

```
chattr +i /etc/resolv.conf      # 不可修改 (即使 root)
chattr +a /var/log/secure        # 仅允许追加
lsattr /etc/resolv.conf
chattr -i /etc/resolv.conf       # 解除
```

6.4 5.4 系统审计 (auditd)

```
# 安装和启用
dnf install audit
systemctl enable --now auditd

# 添加监控规则
auditctl -w /etc/passwd -p wa -k passwd_change
auditctl -w /etc/shadow -p wa -k shadow_access
auditctl -w /etc/sudoers -p wa -k sudoers_change
auditctl -a always,exit -F arch=b64 -S execve -k exec_log

# 持久化: /etc/audit/rules.d/audit.rules

# 查询日志
ausearch -k passwd_change -ts today
ausearch -m USER_LOGIN -sv no        # 失败登录
```

```
aureport --summary
aureport --login --failed
```

6.5 5.5 用户与特权管理

```
# 创建用户
useradd -m -s /bin/bash -G wheel deploy

# 密码策略
chage -M 90 -m 7 -W 14 deploy       # 90 天过期, 最短 7 天, 14 天提醒

# 账户锁定
passwd -l username                    # 锁定
passwd -u username                    # 解锁
faillock --user username              # 查看锁定状态
faillock --user username --reset      # 重置锁定

# 密码复杂度 (/etc/security/pwquality.conf)
minlen = 12
minclass = 3
maxrepeat = 3
```

6.5.1 Sudo 最佳实践

```
# 永远使用 visudo 编辑
visudo

# 最小权限原则
deploy ALL=(ALL) /bin/systemctl restart httpd, /bin/systemctl restart nginx
%dba ALL=(oracle) /opt/oracle/bin/sqlplus

# 审计日志
Defaults logfile="/var/log/sudo.log"
Defaults log_input, log_output
```

6.6 5.6 安全加固清单

类别	措施	优先级
认证	禁用 root SSH 登录	高
认证	启用密钥认证，禁用密码	高
认证	配置 fail2ban 防暴力破解	高
权限	sudo 最小权限	高
防火墙	仅开放必要端口	高
SELinux	保持 Enforcing 模式	中
审计	启用 auditd 监控关键文件	中

类别	措施	优先级
加密	数据分区 LUKS 加密	中
更新	定期安全补丁	中
完整性	部署 AIDE 文件完整性检查	低
扫描	OpenSCAP 合规扫描	低

```
# AIDE 文件完整性
dnf install aide
aide --init
mv /var/lib/aide/aide.db.new.gz /var/lib/aide/aide.db.gz
aide --check

# OpenSCAP 合规扫描
dnf install openscap-scanner scap-security-guide
oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis \
  --results /tmp/scan.xml \
  /usr/share/xml/scap/ssg/content/ssg-rhel9-ds.xml
```

Chapter 7

第六章核心服务管理

本章目标：掌握生产环境中常见服务的配置、管理和排障。

7.1 6.1 systemd 服务管理

```
# 基本操作
systemctl start|stop|restart|reload <service>
systemctl enable|disable <service>
systemctl status <service>
systemctl is-active <service>
systemctl is-enabled <service>

# 查看所有失败服务
systemctl --failed

# 查看服务依赖
systemctl list-dependencies <service>

# 屏蔽服务（防止被其他服务启动）
systemctl mask <service>
systemctl unmask <service>
```

7.1.1 自定义 systemd 服务

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application
After=network.target
Wants=network-online.target

[Service]
Type=simple
User=appuser
Group=appgroup
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/server
Restart=on-failure
RestartSec=5
LimitNOFILE=65535
```

```
[Install]
WantedBy=multi-user.target

systemctl daemon-reload
systemctl enable --now myapp
```

7.2 6.2 Chrony 时间同步

```
# 配置 /etc/chrony.conf
server ntp1.aliyun.com iburst
server time.cloudflare.com iburst

# 常用命令
chronyc sources -v          # 查看时间源
chronyc tracking            # 同步状态
chronyc makestep            # 强制步进同步

# 时区设置
timedatectl
timedatectl set-timezone Asia/Shanghai
timedatectl set-ntp yes
```

7.3 6.3 Rsyslog & journald 日志

7.3.1 Rsyslog

```
# 远程日志（客户端）
# /etc/rsyslog.d/remote.conf
*.* @@log-server:514          # TCP
local0.* /var/log/myapp.log   # 自定义 facility

# 远程日志（服务端）
module(load="imtcp")
input(type="imtcp" port="514")
```

7.3.2 journald

```
journalctl -u sshd -f                    # 跟踪服务日志
journalctl --since "1 hour ago" -p err    # 最近 1 小时错误
journalctl -b -1                          # 上次启动日志
journalctl --disk-usage                   # 日志空间
journalctl --vacuum-size=500M             # 清理到 500M
```

7.3.3 logrotate

```
# /etc/logrotate.d/myapp
/var/log/myapp/*.log {
```

```
    daily
    rotate 30
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
```

7.4 6.4 Web 服务器

7.4.1 Apache

```
dnf install httpd mod_ssl
systemctl enable --now httpd

# 虚拟主机
cat > /etc/httpd/conf.d/mysite.conf << 'EOF'
<VirtualHost *:443>
    ServerName www.example.com
    DocumentRoot /var/www/mysite
    SSLEngine on
    SSLCertificateFile /etc/pki/tls/certs/server.crt
    SSLCertificateKeyFile /etc/pki/tls/private/server.key
</VirtualHost>
EOF

httpd -t && systemctl reload httpd
```

7.4.2 Nginx (替代方案)

```
dnf install nginx
systemctl enable --now nginx

# /etc/nginx/conf.d/mysite.conf
server {
    listen 443 ssl http2;
    server_name www.example.com;
    ssl_certificate     /etc/pki/tls/certs/server.crt;
    ssl_certificate_key /etc/pki/tls/private/server.key;
    root /var/www/mysite;
}

nginx -t && systemctl reload nginx
```

7.5 6.5 NFS

```
# === 服务端 ===
dnf install nfs-utils
systemctl enable --now nfs-server

# /etc/exports
/data/share  10.0.0.0/8(rw,sync,no_root_squash)
exportfs -rav

# === 客户端 ===
mount -t nfs4 server:/data/share /mnt/nfs
# /etc/fstab
server:/data/share  /mnt/nfs  nfs4  defaults,_netdev  0  0

# Autofs 自动挂载
# /etc/auto.master
/mnt/nfs  /etc/auto.nfs  --timeout=300
# /etc/auto.nfs
data  -rw,soft  server:/data/share

# 排错
showmount -e server
nfsstat -c
rpcdebug -m nfs -s all
```

7.6 6.6 Samba (CIFS)

```
dnf install samba samba-client

# /etc/samba/smb.conf
[shared]
    path = /data/samba
    writable = yes
    valid users = @smbgroup
    create mask = 0660

smbpasswd -a username
systemctl enable --now smb nmb

# Linux 客户端挂载
mount -t cifs //server/shared /mnt/smb \
  -o credentials=/root/.smbcred,_netdev
```

7.7 6.7 LDAP / AD 集成

```
# SSSD + realmd
dnf install sssd realmd adcli
realm join --user=admin ad.example.com
realm list

# 验证
id admin@ad.example.com
getent passwd admin@ad.example.com

# 排错
sssctl domain-status ad.example.com
sss_cache -E                # 清除缓存
journalctl -u sssd -f
```

7.8 6.8 KVM 虚拟化

```
# 安装
dnf install qemu-kvm libvirt virt-install
systemctl enable --now libvirtd

# 创建 VM
virt-install --name vm01 --ram 4096 --vcpus 2 \
  --disk path=/var/lib/libvirt/images/vm01.qcow2,size=50 \
  --os-variant rhel9.0 \
  --cdrom /iso/rhel-9.4-x86_64-dvd.iso \
  --network bridge=br0 --graphics vnc

# 管理
virsh list --all
virsh start|shutdown|reboot|console vm01
virsh snapshot-create-as vm01 snap1
virsh snapshot-revert vm01 snap1

# 热添加
virsh setvcpus vm01 4 --live
virsh setmem vm01 8G --live
```

7.9 6.9 Postfix 邮件

```
dnf install postfix
systemctl enable --now postfix

# /etc/postfix/main.cf
myhostname = mail.example.com
relayhost = [smtp.relay.com]:587
```

```
smtp_tls_security_level = encrypt

# 队列管理
postqueue -p          # 查看
postqueue -f          # 刷新
postsuper -d ALL      # 清空
```

Chapter 8

第七章容器与云原生

本章目标：掌握 Podman 容器管理、容器镜像构建、容器网络存储以及 Kubernetes 基础排障。

8.1 7.1 为什么是 Podman?

RHEL 8+ 用 **Podman** 替代 Docker 作为默认容器引擎。核心优势：

特性	Podman	Docker
守护进程	**无** (Daemonless)	需要 dockerd
Root 权限	支持 Rootless	需要 root
兼容性	100% Docker CLI 兼容	–
systemd 集成	原生支持	需额外配置
安全性	更高 (无特权守护进程)	守护进程有 root 权限

```
# 安装
dnf install podman podman-compose

# 验证
podman --version
podman info
```

8.2 7.2 容器基本操作

```
# 拉取镜像
podman pull docker.io/library/nginx:latest
podman pull registry.access.redhat.com/ubi9/ubi:latest

# 运行容器
podman run -d --name web -p 8080:80 nginx
podman run -it --rm ubi9 /bin/bash

# 管理
podman ps -a                    # 列出所有容器
podman logs -f web              # 查看日志
podman exec -it web /bin/bash   # 进入容器
podman stop web
podman rm web
podman rmi nginx
```

```
# 资源限制
podman run -d --name app \
  --memory=512m --cpus=1.5 \
  -p 3000:3000 myapp:latest
```

8.3 7.3 容器镜像构建

8.3.1 Containerfile (等同于 Dockerfile)

```
# Containerfile
FROM registry.access.redhat.com/ubi9/ubi-minimal:latest

LABEL maintainer="admin@example.com"
LABEL version="1.0"

RUN microdnf install -y python3 && \
    microdnf clean all

WORKDIR /app
COPY requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt
COPY . .

EXPOSE 8000
USER 1001

CMD ["python3", "app.py"]

# 构建
podman build -t myapp:1.0 .

# 多阶段构建 (减小镜像体积)
podman build --squash -t myapp:1.0 .

# 推送到 Registry
podman login registry.example.com
podman push myapp:1.0 registry.example.com/myapp:1.0
```

8.3.2 镜像最佳实践

原则	说明
使用最小基础镜像	`ubi-minimal` 替代 `ubi`，Alpine 替代 Ubuntu
合并 RUN 层	减少镜像层数和体积
非 root 用户	`USER 1001`
多阶段构建	编译环境和运行环境分离

原则	说明
`.containerignore`	排除不必要文件

8.4 7.4 容器网络

```
# 默认网络
podman network ls

# 创建自定义网络
podman network create mynet

# 容器加入网络
podman run -d --name db --network mynet postgres:16
podman run -d --name app --network mynet -p 8080:8080 myapp

# 容器间通信（同一网络内用容器名）
# app 容器内: ping db → 可达

# DNS 解析
podman run --dns=8.8.8.8 --name web nginx
```

8.5 7.5 容器存储

```
# Named Volume（推荐）
podman volume create dbdata
podman run -d --name db \
  -v dbdata:/var/lib/postgresql/data \
  postgres:16

# Bind Mount
podman run -d --name web \
  -v /host/path:/container/path:Z \
  nginx

# ⚠ SELinux 注意: 使用 :Z（私有）或 :z（共享）标签

# 查看和管理
podman volume ls
podman volume inspect dbdata
podman volume prune          # 清理未使用的卷
```

8.6 7.6 Rootless 容器

```
# 非 root 用户运行容器
podman run --user 1000:1000 -d nginx

# 配置 UID 映射
cat /etc/subuid    # 查看用户的子 UID 范围
cat /etc/subgid

# Rootless 端口限制: 默认不能绑定 1024 以下端口
# 解决方案:
sysctl net.ipv4.ip_unprivileged_port_start=80
# 或使用端口映射: -p 8080:80
```

8.7 7.7 systemd 集成

```
# 为运行中的容器生成 systemd 服务文件
podman generate systemd --name web --new --files

# 用户级服务 (Rootless)
mkdir -p ~/.config/systemd/user/
cp container-web.service ~/.config/systemd/user/
systemctl --user daemon-reload
systemctl --user enable --now container-web

# 系统级服务
cp container-web.service /etc/systemd/system/
systemctl daemon-reload
systemctl enable --now container-web
```

8.8 7.8 Podman Compose

```
# docker-compose.yml (Podman 兼容)
version: '3'
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:Z
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - dbdata:/var/lib/postgresql/data
```

```yaml
volumes:
  dbdata:
```

```bash
podman-compose up -d
podman-compose logs -f
podman-compose down
```

8.9 7.9 容器排障

```bash
# 查看容器详情
podman inspect web
podman inspect web | jq '.[0].NetworkSettings'

# 查看资源使用
podman stats

# 查看容器内进程
podman top web

# 导出容器文件系统（用于分析）
podman export web > web-filesystem.tar

# 查看构建历史
podman history myapp:1.0

# 常见问题
# 1. 容器启动后立即退出 → 检查 CMD/ENTRYPOINT, 查看 logs
# 2. 网络不通 → 检查 firewalld, podman network inspect
# 3. 权限问题 → SELinux (:Z), 文件权限, User namespace
# 4. 存储空间 → podman system prune -a
```

8.10 7.10 Kubernetes 基础排障

```bash
# 集群状态
kubectl get nodes
kubectl get pods -A
kubectl get events --sort-by='.lastTimestamp'

# Pod 排障
kubectl describe pod <pod>
kubectl logs <pod> [-c container] [-f]
kubectl exec -it <pod> -- /bin/bash

# 常见 Pod 状态
# CrashLoopBackOff → 应用启动失败, 检查 logs
# ImagePullBackOff → 镜像拉取失败, 检查 registry 认证
```

```
# Pending          → 资源不足或调度约束，检查 describe
# OOMKilled        → 内存不足，增加 limits

# 网络排障
kubectl get svc
kubectl get ingress
kubectl run debug --image=busybox --rm -it -- /bin/sh

# 资源查看
kubectl top nodes
kubectl top pods
```

Chapter 9

第八章 Shell 与自动化

本章目标：掌握 Bash 脚本编写、文本处理工具链和自动化最佳实践。

9.1 8.1 Bash 核心速查

9.1.1 变量与参数

语法	功能	示例
`${var:-default}`	空值用默认值	`${DB_HOST:-localhost}`
`${var:=default}`	空值赋默认值	`${LOG_DIR:=/var/log}`
`${#var}`	字符串长度	`${#filename}`
`${var//old/new}`	全局替换	`${path//\//-}`
`${var##*/}`	提取文件名	`/a/b/c.txt` → `c.txt`
`${var%/*}`	提取目录	`/a/b/c.txt` → `/a/b`
`${var%.txt}`	去掉后缀	`file.txt` → `file`

9.1.2 特殊变量

变量	含义
`$0`	脚本名
`$1-$N`	位置参数
`$#`	参数个数
`$@`	所有参数（推荐，正确处理空格）
`$?`	上条命令退出码
`$$`	当前进程 PID
`$!`	最后后台进程 PID

9.1.3 条件测试

```
# 文件测试
[[ -f /etc/passwd ]]     # 文件存在
[[ -d /tmp ]]            # 目录存在
[[ -x /usr/bin/curl ]]   # 可执行
[[ -s /var/log/app.log ]]  # 文件非空

# 字符串
```

```
[[ -n "$var" ]]          # 非空
[[ -z "$var" ]]          # 为空
[[ "$a" == "$b" ]]       # 相等
[[ "$str" =~ ^[0-9]+$ ]] # 正则匹配

# 数值
(( count > 10 ))
(( a == b ))
```

9.1.4 I/O 重定向

```
cmd > file               # 覆盖
cmd >> file              # 追加
cmd 2> error.log         # 错误重定向
cmd &> all.log           # 合并所有输出
cmd > /dev/null 2>&1     # 丢弃所有输出
```

9.2 8.2 循环与函数

```
# for 循环
for svc in httpd nginx mysqld; do
    systemctl is-active "$svc" &>/dev/null \
      && echo "□  $svc: UP" \
      || echo "□  $svc: DOWN"
done

# while 读文件
while IFS= read -r line; do
    echo "Processing: $line"
done < input.txt

# 函数
check_disk() {
    local threshold=${1:-80}
    df -h | awk -v t="$threshold" \
      'NR>1 && int($5)>t {printf "⚠   %s: %s\n", $6, $5}'
}
check_disk 90
```

9.3 8.3 生产脚本模板

```
#!/bin/bash
set -euo pipefail    # 严格模式

readonly SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
readonly LOG="/var/log/$(basename "$0" .sh).log"
readonly LOCK="/tmp/$(basename "$0" .sh).lock"
```

```
log()  { echo "[$(date '+%F %T')] $*" | tee -a "$LOG"; }
die()  { log "□  ERROR: $*" >&2; exit 1; }
info() { log "□    $*"; }

# 防止重复运行
exec 200>"$LOCK"
flock -n 200 || die "Script is already running"

# 参数检查
[[ $# -ge 1 ]] || die "Usage: $0 <arg>"

# 清理函数
cleanup() { rm -f /tmp/$$_*; log "Cleanup done."; }
trap cleanup EXIT
trap 'die "Caught signal at line $LINENO"' ERR

info "Starting..."
# ... 业务逻辑 ...
info "Done."
```

9.4 8.4 grep

```
grep -rn "error" /var/log/          # 递归搜索 + 行号
grep -i "warning" *.log             # 忽略大小写
grep -E "error|fatal|panic" app.log  # 扩展正则 (OR)
grep -v "^#" /etc/ssh/sshd_config  # 排除注释
grep -c "ORA-" alert.log            # 计数
grep -l "TODO" src/*.py             # 仅输出文件名
grep -A 3 -B 1 "FATAL" app.log     # 上下文
grep -P '\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}' access.log  # Perl 正则匹配 IP
```

9.5 8.5 find

```
find /var/log -name "*.log" -mtime +30 -delete   # 删除 30 天前日志
find / -size +100M -type f -exec ls -lh {} \;    # 大文件
find / -perm -4000 -type f -ls                   # SUID 文件 (安全审计)
find /data -name "*.bak" -print0 | xargs -0 rm   # 安全处理空格文件名
find / -nouser -o -nogroup                        # 无主文件
```

9.6 8.6 awk

```
# 字段提取
awk -F: '{print $1, $7}' /etc/passwd

# 条件过滤
```

```
awk '$5 > 90 {print $1, $5"%"}' <(df -h)

# 统计
awk '{sum+=$1} END {print "Total:", sum}' numbers.txt
awk '{count[$1]++} END {for(k in count) print k, count[k]}' access.log

# 格式化输出
awk -F: 'BEGIN {printf "%-20s %s\n","USER","SHELL"} \
  {printf "%-20s %s\n",$1,$7}' /etc/passwd
```

9.7 8.7 sed

```
sed 's/old/new/g' file.txt           # 替换
sed -i 's/old/new/g' file.txt        # 原地修改
sed -n '10,20p' file.txt             # 打印行范围
sed '/^#/d; /^$/d' config.conf       # 删除注释和空行
sed -i '/pattern/a\new line' file.txt  # 匹配行后添加
```

9.8 8.8 实用管道组合

```
# Top 10 最大目录
du -sh /* 2>/dev/null | sort -rh | head -10

# 统计日志错误类型
grep -oP 'ORA-\d+' alert.log | sort | uniq -c | sort -rn

# IP 访问排行
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head

# 实时日志监控 + 过滤
tail -f /var/log/messages | grep --color -iE "error|warn|fail"

# 批量文件重命名
for f in *.JPEG; do mv "$f" "${f%.JPEG}.jpg"; done

# 并行执行
cat hosts.txt | xargs -P 10 -I {} ssh {} "hostname; uptime"
```

9.9 8.9 Ansible 快速入门

```
# 安装
dnf install ansible-core

# Ad-hoc 命令
ansible all -m ping -i inventory.ini
```

```
ansible web -m shell -a "uptime" -i inventory.ini
ansible db -m yum -a "name=postgresql state=latest" -i inventory.ini
```

9.9.1 Playbook 示例

```yaml
# deploy-web.yml
---
- name: Deploy Web Server
  hosts: web
  become: yes
  tasks:
    - name: Install Nginx
      dnf:
        name: nginx
        state: latest

    - name: Start and enable Nginx
      systemd:
        name: nginx
        state: started
        enabled: yes

    - name: Open firewall port
      firewalld:
        service: http
        permanent: yes
        state: enabled
        immediate: yes
```

```
ansible-playbook deploy-web.yml -i inventory.ini
```

Chapter 10

第九章性能分析与调优

本章目标：建立系统化的性能分析方法论，掌握 CPU、内存、IO、网络各维度的诊断和调优。

10.1 9.1 性能分析方法论

10.1.1 USE 方法 (Brendan Gregg)

对每个资源检查三个指标：**Utilization (利用率) + Saturation (饱和度) + Errors (错误)**

资源	利用率 (U)	饱和度 (S)	错误 (E)
CPU	`mpstat, top`	运行队列长度 (`vmstat r`)	`dmesg, perf`
内存	`free -h`	Swap 使用, OOM	`dmesg` OOM 日志
磁盘 IO	`iostat %util`	`await`, 队列深度	`smartctl`, IO error
网络	`sar -n DEV`	TCP retransmit	`ip -s link` 错误计数

10.1.2 快速排障流程

```
1. uptime              → 负载趋势
2. dmesg -T | tail     → 内核错误
3. vmstat 1 5          → CPU/内存/IO 概览
4. iostat -dxm 1 5     → 磁盘 IO
5. free -h             → 内存
6. sar -n DEV 1 5      → 网络
7. top -bn1 | head -20 → 进程 CPU/内存排行
8. ss -tuln            → 监听端口
```

10.2 9.2 CPU 分析

```
# 总体负载
uptime
# load average: 2.50, 2.30, 2.10
# 数值/CPU 核心数 > 1 表示过载

# 每核心统计
mpstat -P ALL 2 5

# 进程级
top -H -p <pid>            # 查看线程
```

```
pidstat -u 2 5

# 内核级分析
perf top                    # 实时热点函数
perf record -g -p <pid>     # 采样
perf report                 # 分析

# CPU 频率
cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
cpupower frequency-info
cpupower frequency-set -g performance   # 高性能模式
```

10.3 9.3 内存分析

```
# 概览
free -h
cat /proc/meminfo | grep -E "MemTotal|MemFree|MemAvailable|Cached|Buffers|SwapTotal|SwapFree"

# 进程内存
ps aux --sort=-%mem | head -10
pmap -x <pid>

# NUMA
numactl --hardware
numastat

# Swap 分析
swapon -s
vmstat 2 5    # 关注 si/so 列 (swap in/out)
# si/so 持续 > 0 说明内存不足

# OOM Killer 日志
dmesg -T | grep -i "oom\|out of memory"
journalctl -k | grep -i oom
```

10.4 9.4 磁盘 IO 分析

```
# iostat 详细
iostat -dxm 2 5
```

10.4.1 iostat 关键指标

指标	含义	告警阈值
`%util`	设备繁忙率	>70% 需关注
`await`	平均 IO 延迟 (ms)	SSD >5ms, HDD >20ms 需关注

指标	含义	告警阈值
r_await/w_await	读/写延迟	差异大说明某方向有瓶颈
avgqu-sz	平均队列深度	>2 需关注
r/s + w/s	IOPS	超过设备规格说明瓶颈

```
# IO 进程排行
iotop -oP

# blktrace 深度分析
blktrace -d /dev/sda -o trace
blkparse -i trace.blktrace.0 | head -50
```

10.4.2 IO 调度器

调度器	适用场景
mq-deadline	数据库 (RHEL 8/9 默认)
none / noop	SSD / NVMe / 虚拟化 (底层已优化)
bfq	桌面/交互式应用

```
# 查看当前调度器
cat /sys/block/sda/queue/scheduler

# 临时修改
echo mq-deadline > /sys/block/sda/queue/scheduler

# 持久化 (GRUB)
# elevator=mq-deadline
```

10.5 9.5 TuneD 自动调优

```
# 查看可用 profile
tuned-adm list

# 推荐 profile
tuned-adm profile throughput-performance    # 高吞吐
tuned-adm profile latency-performance       # 低延迟
tuned-adm profile virtual-guest             # 虚拟机
tuned-adm profile network-throughput        # 网络优化

# 查看当前
tuned-adm active
tuned-adm recommend
```

10.6 9.6 资源限制

10.6.1 ulimits

```
# 查看当前限制
ulimit -a

# /etc/security/limits.conf
oracle  soft  nofile  65535
oracle  hard  nofile  65535
oracle  soft  nproc   65535
oracle  hard  nproc   65535
*       soft  core    unlimited

# systemd 服务限制
# 在 [Service] 段添加:
LimitNOFILE=65535
LimitNPROC=65535
```

10.6.2 cgroups v2

```
# 查看 cgroup 版本
stat -f /sys/fs/cgroup/ | grep Type
# cgroup2fs = v2

# systemd slice 查看
systemd-cgtop

# 限制服务资源
systemctl set-property myapp.service CPUQuota=200%
systemctl set-property myapp.service MemoryMax=2G
```

Chapter 11

第十章诊断工具与排障方法论

本章目标：掌握企业级诊断工具和系统化排障方法。

11.1 10.1 Sosreport

```
# 收集系统诊断信息包
sosreport

# 仅收集特定插件
sos report --only-plugins=networking,filesys,logs

# 输出: /var/tmp/sosreport-<hostname>-<date>.tar.xz
# 用于提交给技术支持团队分析
```

11.2 10.2 PCP (Performance Co-Pilot)

```
# 安装
dnf install pcp pcp-system-tools pcp-gui
systemctl enable --now pmcd pmlogger

# 实时监控
pmstat -s 5
pmval -T 10sec disk.dev.read_bytes
pcp atop                        # 类似 atop 的 PCP 版本

# 历史数据回放
pmchart                         # GUI
pmrep -t 5sec -s 30 disk.dev.total kernel.all.cpu.idle
```

11.3 10.3 eBPF 工具 (bcc / bpftrace)

eBPF 是 Linux 内核中的革命性技术，允许在内核中安全运行自定义程序，是新一代性能分析的核心。

```
# 安装
dnf install bcc-tools bpftrace

# 常用 bcc 工具 (位于 /usr/share/bcc/tools/)
execsnoop          # 跟踪新进程创建
```

```
opensnoop          # 跟踪文件打开
biolatency         # 块 IO 延迟直方图
tcplife            # TCP 连接生命周期
tcpretrans         # TCP 重传
cachestat          # 页缓存命中率
ext4slower 1       # 慢 ext4 操作 (>1ms)

# bpftrace 一行式
bpftrace -e 'tracepoint:syscalls:sys_enter_openat { printf("%s %s\n", comm, str(args->filename
bpftrace -e 'tracepoint:block:block_rq_complete { @[args->rwbs] = count(); }'
```

11.4 10.4 strace 系统调用跟踪

```
# 跟踪进程
strace -p <pid>
strace -c -p <pid>              # 统计摘要
strace -e trace=network -p <pid>  # 仅网络调用
strace -e trace=file -p <pid>    # 仅文件操作
strace -f -o /tmp/trace.log command  # 跟踪子进程
```

11.5 10.5 排障方法论

11.5.1 五步排障法

```
1⃣ 收集信息  → 症状是什么？什么时候开始？最近有什么变更？
2⃣ 形成假设  → 基于症状和经验，列出最可能的原因（Top 3）
3⃣ 测试验证  → 逐一验证假设，从最简单的开始
4⃣ 实施修复  → 确认根因后修复，记录操作步骤
5⃣ 记录总结  → 写 Post-mortem，更新知识库，防止复发
```

11.5.2 常见故障排查路径

系统无法启动

```
检查 GRUB → 检查内核 → 检查 initramfs → 检查 fstab → 检查 systemd
```

磁盘空间满

```
df -h → du -sh /* | sort -rh → find / -size +100M → lsof | grep deleted
```

网络不通

```
ip a → ping gateway → ping DNS → dig → traceroute → ss/tcpdump → firewall-cmd
```

进程 CPU 100%

```
top → strace -c -p <pid> → perf top -p <pid> → 检查应用日志
```

OOM Killed

```
dmesg -T | grep oom → 分析 /proc/<pid>/status → 检查 cgroup 限制 → 优化应用或增加内存
```

11.6 10.6 日志分析速查

```
# 系统日志
journalctl -p err -b              # 当前启动以来的错误
journalctl --since "2 hours ago"

# 认证日志
grep "Failed password" /var/log/secure | tail -20

# 内核日志
dmesg -T --level=err,warn | tail -50

# 服务日志
journalctl -u <service> --since today

# 日志聚合分析
grep -E "error|fail|critical" /var/log/messages | \
  awk '{print $5}' | sort | uniq -c | sort -rn | head
```

Chapter 12

附录 A 速查表

12.1 A.1 systemctl 速查

命令	功能
`systemctl start/stop/restart svc`	启停服务
`systemctl enable/disable svc`	开机自启
`systemctl status svc`	查看状态
`systemctl --failed`	列出失败服务
`systemctl list-units --type=service`	列出所有服务
`systemctl daemon-reload`	重载 unit 文件
`systemctl mask/unmask svc`	屏蔽/解除屏蔽

12.2 A.2 文件系统操作速查

操作	XFS	Ext4
创建	`mkfs.xfs`	`mkfs.ext4`
扩展	`xfs_growfs /mount`	`resize2fs /dev/x`
修复	`xfs_repair /dev/x`	`e2fsck -f /dev/x`
信息	`xfs_info /mount`	`tune2fs -l /dev/x`
碎片整理	`xfs_fsr`	`e4defrag`

12.3 A.3 网络诊断速查

工具	用途
`ss -tuln`	查看监听端口
`ip a`	查看 IP 地址
`ip r`	查看路由表
`mtr <host>`	持续 traceroute
`dig <domain>`	DNS 查询
`tcpdump -i eth0`	抓包
`ethtool eth0`	网卡信息
`nmap -sT <host>`	端口扫描

12.4 A.4 性能工具速查

工具	用途	关键参数
top / htop	进程监控	top -H -p <pid>
vmstat	系统概览	vmstat 2 10
iostat	磁盘 IO	iostat -dxm 2
mpstat	每 CPU 统计	mpstat -P ALL 2
sar	历史数据	sar -u/-r/-d 2 10
pidstat	进程级统计	pidstat -d 2
perf	内核级分析	perf top, perf record
strace	系统调用	strace -c -p <pid>

12.5 A.5 容器速查

操作	命令
运行容器	podman run -d --name x -p 8080:80 image
查看日志	podman logs -f x
进入容器	podman exec -it x /bin/bash
构建镜像	podman build -t name:tag .
查看资源	podman stats
清理	podman system prune -a
生成 systemd	podman generate systemd --name x --new

12.6 版权声明

Linux Ops Fieldbook v1.0 (c) 2026 Bo Wang. All rights reserved.

本书内容基于作者多年生产环境实战经验编写。所有命令和配置均基于公开的 Linux 发行版文档和社区最佳实践。

Last updated: March 2026 | Compatible with RHEL/Rocky/Alma/Oracle Linux 8.x & 9.x